

Name: Huynh Ha Giang Phan
Student Number: a1901146

1. Overview:

Submission 2 investigates shared-memory parallelisation of the N-body simulation using OpenMP. The objective is to parallelise the computationally expensive force calculations while maintaining correct updates to particle forces, positions and velocities.

Part 2A focuses on synchronization. Since several OpenMP threads may calculate different particle interactions concurrently, multiple threads can attempt to update the same force entry at the same time. Two synchronization approaches were implemented and compared: an OpenMP critical region and explicit OpenMP locks.

Part 2B focuses on performance and scheduling. Four implementations were tested: Basic, Reduced Default, Reduced Forces Cyclic, and Reduced All Cyclic. Their runtime, speedup and parallel efficiency were measured over several thread counts.

2. Part 2A — Synchronization:

2.1 Race condition:

The shared-force implementation exploits Newton's third law by calculating the interaction between particles i and j once and applying equal and opposite contributions: $F_i \leftarrow F_i + F_{ij}$; $F_j \leftarrow F_j - F_{ij}$

Parallelising the outer force-calculation loop introduces a race condition because different iterations can update the same element of the shared forces array. For example, one thread processing interaction (0, 2) may update `forces[2]` while another thread processing (1, 2) also updates `forces[2]`. An operation such as `forces[k][X] -= f_part_k[X]`; is a read-modify-write operation rather than a single indivisible update. Without synchronization, simultaneous accesses can therefore cause an update to be lost and produce incorrect forces.

The force calculation must also remain separated from the position/velocity update. All force calculations for a timestep must finish before any thread starts updating particles using those forces. The implicit barriers at the end of the OpenMP for regions provide this synchronization between the reset, force-calculation and particle-update stages.

2.2 Data sharing with default(none):

The main OpenMP parallel region uses `default(none)`, requiring the sharing status of variables to be specified explicitly. This makes the parallel data dependencies clearer and reduces the chance that a variable is unintentionally shared.

The particle array `curr` and force array `forces` are **shared**, since all threads operate on the same simulation state. Simulation parameters such as the number of particles, number of timesteps, timestep size and output frequency are also shared because they are read by all threads and do not independently change.

Loop-control variables such as `part` and `step` are private to each thread where required. In the lock implementation, the lock array is also shared because every thread must synchronize through the same set of locks.

The OpenMP work-sharing loops distribute the particle and interaction calculations among the threads, while their implicit barriers ensure that the stages of each timestep occur in the correct order.

2.3 Critical-section implementation:

The first implementation protects the shared force accumulation using an OpenMP critical region. Once a thread calculates the force for a particle pair, it enters the critical section before modifying the shared force array. Conceptually:

```
#pragma omp critical
{
    forces[part][X] += f_part_k[X];
    forces[part][Y] += f_part_k[Y];

    forces[k][X] -= f_part_k[X];
    forces[k][Y] -= f_part_k[Y];
}
```

This removes the race condition because only one thread can execute the protected updates at a time. The design is simple and makes correctness relatively easy to reason about. However, the critical region uses coarse-grained synchronization: even two threads updating completely different particles cannot perform their protected updates simultaneously.

2.4 Lock-based implementation:

The second implementation uses an array containing one OpenMP lock for each particle. Before updating a particle's force entry, a thread acquires that particle's lock and releases it immediately after completing the update. For example:

```
omp_set_lock(&locks[part]);

forces[part][X] += f_part_k[X];
forces[part][Y] += f_part_k[Y];

omp_unset_lock(&locks[part]);
```

The same procedure is used when updating the force for particle `k`.

This provides finer-grained synchronization than a single critical region. Threads operating on different force entries can proceed concurrently, while conflicting updates to the same particle remain mutually exclusive.

The implementation acquires and releases one particle lock at a time instead of holding two particle locks simultaneously. This simplifies the locking scheme and avoids circular lock dependencies that could otherwise cause deadlock. Locks are initialized before entering the parallel computation and destroyed after the parallel region finishes.

Therefore, both implementations solve the same race condition, but their synchronization granularity differs: the critical implementation serializes the complete shared-force update, whereas the lock implementation protects individual particle force entries.

3. Part 2A — Validation:

Correctness was checked by running the implementations with multiple OpenMP thread counts and comparing the resulting particle states. Tests used generated initial conditions and included 1, 2, 4 and 8 threads.

For the lock implementation, a test with 16 particles, five timesteps and $\Delta t = 0.01$ produced consistent displayed states across the tested thread counts. For example, at $t = 0.05$, particle 0 reached $x = 5.273 \times 10^1$ with x-velocity 2.636×10^3 in the tested runs.

A single-particle case was also tested. Since there is no second particle, no gravitational pair interaction exists; the output showed zero x acceleration while the particle continued according to its initial y velocity, providing a useful boundary-case check.

Both normal-output and NO_OUTPUT builds were tested. The latter confirmed that the synchronization implementation also operated correctly in the configuration used for performance-oriented execution.

4. Experimental Environment and Methodology:

Performance testing was conducted on an Apple M3 with 8 CPU cores (4 performance and 4 efficiency cores) and 8 GB memory. The OpenMP programs were compiled using Homebrew GCC 16, with -O2, -fopenmp, and -DNO_OUTPUT. Disabling simulation output prevented repeated console I/O from dominating the timing measurements.

The experiment used 2,000 particles, 200 timesteps and $\Delta t = 0.01$. Four implementations were tested: Basic, Reduced Default, Reduced Forces Cyclic and Reduced All Cyclic. Thread counts of 1, 4, 8, 16 and 32 were used. Each configuration was executed five times, giving 100 measurements in total, and mean elapsed runtime was used for comparison.

Speedup and efficiency were calculated separately for each implementation using its own mean one-thread runtime: $Sp = T1/Tp$; $Ep = Sp/p$.

The 16- and 32-thread configurations exceed the eight available CPU cores and therefore represent oversubscribed execution.

5. Results:

Table 1. Mean execution time over five runs (seconds).

Threads	Basic	Reduced Default	Forces Cyclic	All Cyclic
---------	-------	-----------------	---------------	------------

1	0.8912	1.0870	1.1926	1.1453
4	0.2874	0.2445	0.2933	0.1984
8	0.2582	0.274	0.2761	0.2435
16	0.3603	0.2676	0.2453	0.2468
32	0.3374	0.3537	0.2330	0.2369

The corresponding speedups were:

Threads	Basic	Reduced Default	Forces Cyclic	All Cyclic
1	1.000	1.000	1.000	1.000
4	3.101	4.446	4.066	5.774
8	3.452	3.954	4.320	4.703
16	2.474	4.062	4.863	4.641
32	2.641	3.073	5.118	4.834

Figure 1. Speedup versus number of OpenMP threads.

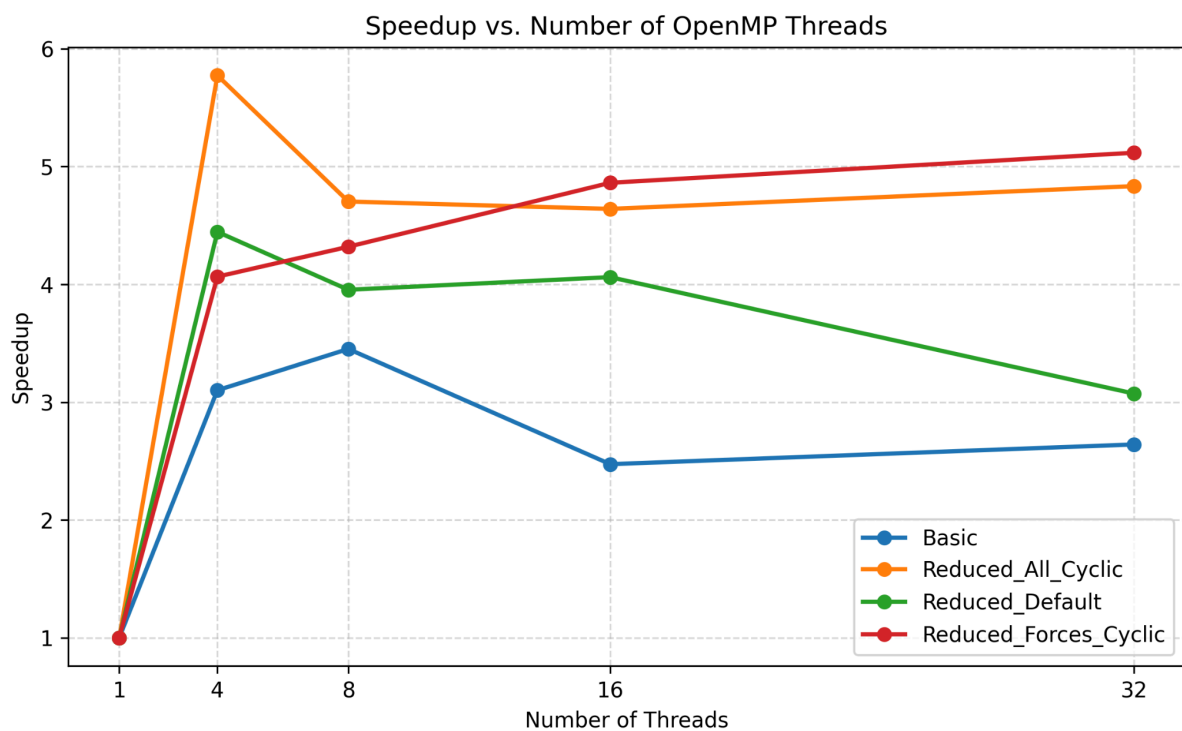
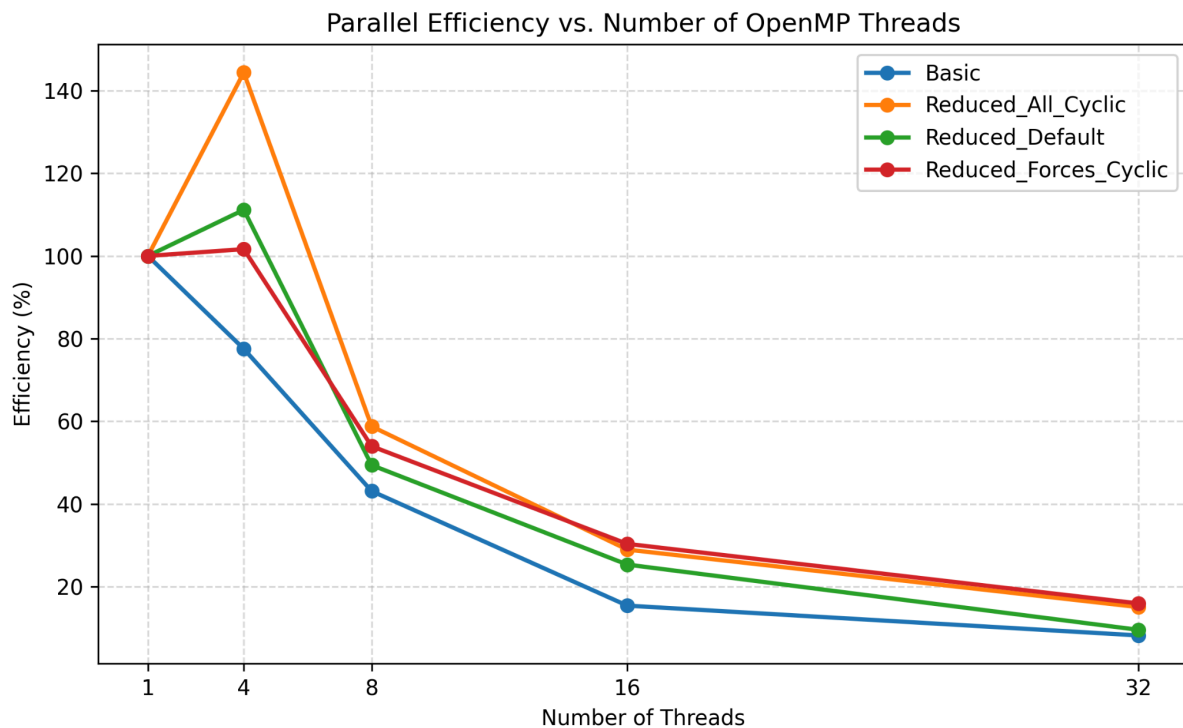


Figure 2. Parallel efficiency versus number of OpenMP threads.



6. Performance Analysis:

The Basic implementation had the lowest one-thread runtime at 0.8912 s, despite the Reduced implementations exploiting force symmetry to avoid calculating each pair interaction twice. This indicates that reducing the number of force calculations does not automatically reduce total runtime. The reduced OpenMP approach introduces additional overhead associated with thread-local force storage, initialization and combination.

Parallel execution substantially improved runtime initially. Basic decreased from 0.8912 s at one thread to 0.2582 s at eight threads, corresponding to a speedup of 3.452. Its performance then deteriorated to 0.3603 s at 16 threads and 0.3374 s at 32 threads.

Scheduling had a substantial effect on the Reduced implementations. The reduced force loop has an inherently decreasing amount of work: early outer-loop iterations interact with many remaining particles, while later iterations interact with fewer. With default static partitioning, contiguous groups of these iterations can therefore give different amounts of work to different threads.

Cyclic scheduling distributes loop iterations among threads in a round-robin pattern, mixing expensive early iterations with cheaper later iterations. This improves load balance in the force-calculation stage. At 32 threads, Reduced Forces Cyclic recorded 0.2330 s, compared with 0.3537 s for Reduced Default.

Reduced All Cyclic recorded the lowest mean runtime at four and eight threads, with 0.1984 s and 0.2435 s, respectively. Reduced Forces Cyclic was slightly faster at 16 and 32 threads, recording 0.2453 s and 0.2330 s. This suggests that applying cyclic scheduling to the imbalanced force loop provides the important scheduling benefit, while cyclic scheduling of every loop is not necessarily beneficial at all thread counts.

Efficiency declined as the number of threads increased. At eight threads, efficiencies ranged from approximately 43% to 59%; by 32 threads they ranged from approximately 8% to 16%. The 16- and 32-thread configurations oversubscribe the eight-core test system, so increasing the OpenMP thread count does not provide an equivalent increase in hardware execution resources. Parallel overhead and synchronization also become increasingly important as additional threads are introduced.

Several Reduced results at four threads produced calculated speedups above four and therefore efficiencies above 100%. These measurements should be interpreted cautiously. They may be influenced by cache behaviour, OpenMP runtime effects, the heterogeneous M3 architecture and run-to-run timing variability. In addition, because speedup uses the measured one-thread mean as its denominator, variation in that baseline directly affects the calculated speedup. The measurements were retained because the experimental methodology specified mean execution time rather than selectively removing slower or faster runs.

7. Conclusion

The OpenMP implementation demonstrated that synchronization is essential when parallel threads update the shared N-body force array. Both the critical-section and per-particle lock implementations prevented conflicting force updates, with locks providing finer-grained synchronization.

The performance experiment also showed that parallel performance depends on more than the number of force calculations. Although Basic was fastest with one thread, the reduced implementations became competitive under parallel execution. Cyclic scheduling improved load distribution for the uneven reduced-force loop, with All Cyclic performing strongly at four and eight threads and Forces Cyclic producing the lowest measured runtimes at 16 and 32 threads. Scaling efficiency decreased at larger thread counts, particularly once the number of OpenMP threads exceeded the available hardware cores.